iOS Developer
Technical Exercise: Mini Finance

A practical assessment covering SwiftUI, iOS architecture, Git, Figma implementation, MCP, and responsible AI-assisted development.

Format	Suggested time
Take-home exercise + technical discussion	3-5 hours

The brief

Build a small personal-finance iOS app named **Mini Finance**. The app lets a user review account activity, open transaction details, add a transaction, and mark a transaction as reviewed. Use mock data only - do not connect to real financial services.

Design reference

Use the public Figma Community file **Mini Finance UI Kit** by Emmanuel Ikechukwu as the visual reference. Implement the selected screens faithfully while making sensible iOS and accessibility adaptations.

Figma file: https://www.figma.com/community/file/1477789486167752692

What to submit

- A Git repository containing the Xcode project and README.
- A pull request from a feature branch into main, with a clear description and screenshots.
- AI_NOTES.md answering the MCP and AI questions on page 3.

Please keep the scope focused. We value clear engineering decisions and a working, polished result over extra features.

Product and UI requirements

Implement the following screens from the Mini Finance UI Kit. You may choose the closest matching frames if names differ.

Required screens

1. Dashboard	Show account balance, high-level spending information, and recent transactions.
2. Transactions	Show a scrollable list with category, merchant/title, date, and amount.
3. Transaction detail	Show complete transaction information and a visible reviewed/unreviewed state.
4. Budget or category	Show spending by category or a budget progress view based on the design reference.
5. Add transaction	Present a form/sheet that validates basic inputs and adds a local transaction.

Functional requirements

• Use SwiftUI, NavigationStack, and appropriate navigation patterns.

• Load transaction data asynchronously using async/await.

• Support loading, error, empty, and populated states.

• Add pull-to-refresh to the transaction list or dashboard.

• Allow a transaction to be marked as reviewed and persist that state across launches.

• Use a local JSON fixture or mock service; document how to switch to a real API later.

• Support light/dark mode, Dynamic Type, and VoiceOver labels for important controls.

Figma-to-code expectations

• Extract repeated visual decisions into reusable tokens: color, typography, spacing, corner radius, and shadows where appropriate.

• Create reusable SwiftUI components, such as a balance card, transaction row, category badge, and primary action button.

• Do not hardcode layouts solely for one device. Validate at least one smaller iPhone size and a larger Dynamic Type size.

• Use SF Symbols or legally usable assets when an exported design asset is unavailable.

Engineering requirements

Architecture

• Define a TransactionService protocol and inject the implementation into the presentation layer.

• Keep data loading and business logic out of SwiftUI view bodies.

• Use a testable view model or equivalent observable presentation model.

• Explain state ownership choices, including any use of @State, @Binding, @StateObject, @Observable, or environment injection.

• Ensure UI-facing state changes occur on the main actor.

• Persist added transactions and review status using SwiftData, Core Data, or a justified lightweight local approach.

Testing

• Include at least two automated unit tests for the presentation model/service boundary.

• Cover a successful transaction load and either an error or empty state.

• Describe manual tests for accessibility, dark mode, and persistence in the README.

Git workflow

• Start from main and work on a feature branch.

• Make 4-6 small, meaningful commits. Avoid a single final 'everything' commit.

• Open a PR into main. Include a summary, implementation choices, screenshots, test results, and known limitations.

• Do not commit secrets, personal Xcode user data, or derived build artifacts.

MCP and AI

Create **AI_NOTES.md** (roughly 300-500 words) answering:

• How could an AI assistant use MCP to access the Figma design, GitHub repository/issues, and product or API documentation?

• What minimum permissions should each MCP tool receive? Which actions should require explicit approval?

• Which tasks could AI usefully assist with in this exercise?

• How would you validate AI-generated Swift code before merging it? Consider correctness, privacy, security, accessibility, tests, and licensing.

Important: never place API keys, tokens, personal financial data, or other secrets into an AI prompt, repository, or test fixture.

Evaluation and discussion

Evaluation rubric

Area	Weight	What we look for
SwiftUI and iOS	35%	State ownership, async/await, navigation, separation of concerns, persistence.
Figma and UI quality	20%	Faithful but idiomatic implementation, reusable components, accessibility, adaptability.
Testing and reliability	15%	Useful automated tests, failure states, pragmatic validation.
Git and communication	15%	Safe workflow, understandable commit history, clear PR and README.
MCP and AI judgment	15%	Least privilege, review discipline, privacy and security awareness.

Technical discussion prompts

• Which Figma values did you promote into design tokens, and why?

• Why did you choose your SwiftUI property wrappers and where is state owned?

• How would you add pagination, caching, and offline support?

• How would you avoid UI updates occurring outside the main actor?

• Show one commit that captures an engineering decision. What alternatives did you consider?

• If an AI agent had GitHub write access, what could go wrong? What controls would you put in place?

Optional extensions

These are not required and should not displace the core work: transaction search/filtering, chart accessibility descriptions, snapshot tests, a small Figma-code token mapping document, or a local-only AI-style transaction categorization stub.

Thank you. Be ready to spend 10-15 minutes demonstrating the app and discussing your implementation choices.